Lab 1 · Modelado OLTP → OLAP

SI7006 Almacenamiento y Procesamiento de Grandes Datos · Maestría en Ingeniería de Datos · EAFIT 2026-2 Caso: PayTrack Colombia **Equipo:** Santiago Vélez Casallas · Luis Alberto Ortiz Quiroga **Repositorio:** https://github.com/santiagovelcasallas/big-data-labs-2026-2

> Borrador en Markdown; se entrega en PDF. Enunciado: *"Informe en PDF (máx. 3 páginas): diagnóstico del OLTP (Parte A), decisiones de diseño (Parte B) y comparación de rendimiento con planes de ejecución (Parte D)"*. Las evidencias citadas están en `labs/01-modelado/evidencias/`.

1. Diagnóstico del OLTP (Parte A)

Se siguieron los pasos 1 a 4 del enunciado: PostgreSQL 16 en Docker, 50.000 transacciones cargadas al esquema `oltp` (3FN, 569 páginas de 8 kB) y `EXPLAIN ANALYZE` de las tres consultas de `sql/oltp_problems.sql`, después de `ANALYZE`. Como sugiere el enunciado, también se midió con 500.000 transacciones (5.680 páginas). La proyección usa *500K transacciones diarias durante un año*: unos 182,5 M de filas, cerca de 16 GB de heap (365 veces la prueba de 500.000). Las proyecciones son lineales y son cotas inferiores, con datos en memoria.

Consulta	50.000	500.000	Año (proyección)
Q1: Monto total por categoría de comercio (últimos 30 días)	9,6 ms	52,0 ms	≥ 19 s
Q2: Tasa de fraude por segmento de cliente	17,6 ms	80,9 ms	≥ 30 s
Q3: Volumen de transacciones por día de la semana y hora (últimos 7 días)	4,9 ms	50,9 ms	≥ 4,6 s

Q1: Monto total por categoría de comercio (últimos 30 días)

¿Qué operaciones dominan el plan (scans, joins, sorts)? El Bitmap Heap Scan sobre `transactions` (4,2 de 9,6 ms): el índice de `created_at` halla 16.646 filas, pero se visitan las 569 páginas (`exact=569`). Le sigue el Hash Join con `merchants`. Con 500.000 el optimizador cambia a *Parallel Seq Scan*.
¿Por qué el diseño en 3FN obliga a hacer ese trabajo? `category` depende de `merchant_id` y vive en `merchants`: agrupar por categoría exige un join por transacción. La fecha es un `TIMESTAMP` sin atributos de periodo.
¿Qué pasaría con estos tiempos con 500K transacciones diarias durante un año? La ventana tendría unos 15 M de filas. Como con 500.000 ya se lee la tabla completa, serían unos 16 GB por ejecución: ≥ 19 s (52 ms × 365).

Q2: Tasa de fraude por segmento de cliente

¿Qué operaciones dominan el plan (scans, joins, sorts)? El Hash Join: el Seq Scan de las 50.000 filas cuesta 3,1 ms y el join con `customers` lleva el acumulado a 11,7 ms (2,8 veces la lectura), para producir 3 grupos.
¿Por qué el diseño en 3FN obliga a hacer ese trabajo? `segment` vive en `customers`, así que hay que unir toda la historia de transacciones. Además el 3FN guarda solo el segmento actual: si un cliente cambia de segmento, sus fraudes anteriores se atribuyen al nuevo.
¿Qué pasaría con estos tiempos con 500K transacciones diarias durante un año? Es la que peor escala: recorre la tabla completa y crece cada día aunque la pregunta no cambie. Serían ≥ 30 s (80,9 ms × 365). El paralelismo (4,6× con 10× datos) está limitado por el número de workers.

Q3: Volumen de transacciones por día de la semana y hora (últimos 7 días)

¿Qué operaciones dominan el plan (scans, joins, sorts)? El Sort por `EXTRACT(DOW)` y `EXTRACT(HOUR)` (270 kB) y la lectura de las 569 páginas para 3.735 filas. El optimizador estima 3.776 grupos para 168 reales. Con 500.000 crece 10,5×: el plan no se paraleliza.
¿Por qué el diseño en 3FN obliga a hacer ese trabajo? Día y hora se derivan de `created_at`; guardarlos sería redundante. Se recalculan por fila en cada consulta, sin índices ni estadísticas posibles.
¿Qué pasaría con estos tiempos con 500K transacciones diarias durante un año? La ventana tendría unos 3,5 M de filas (90 veces la de 500.000): ≥ 4,6 s. El Sort pediría unos 285 MB, mucho más que el `work_mem` por defecto (4 MB), así que pasaría a ordenamiento en disco.

Conclusión. El 3FN optimiza registrar pagos sin redundancia. A cambio, cada consulta analítica reconstruye el contexto: joins (Q1, Q2), atributos de calendario calculados (Q3) y lecturas amplias. Además se pierde la historia de los atributos que cambian (Q2), y esas lecturas compiten con la escritura transaccional.

2. Decisiones de diseño (Parte B)

Proceso de negocio y grano de la tabla de hechos.
El proceso es el **procesamiento de pagos**: cada intento de pago de un cliente en un comercio con un método de pago, aprobado o rechazado. El grano es **una fila por transacción** de `oltp.transactions`, el nivel más atómico: las tres preguntas se responden agregando. `transaction_id` es la PK de `fact_transactions` y una *dimensión degenerada*: identifica la transacción, pero no tiene atributos propios.

Dimensiones y los atributos de cada una. ¿Qué datos del OLTP *no* deben llegar al data warehouse?

Dimensión	Surrogate key	Natural key	Atributos
`dim_date` (dada)	`date_key`	`full_date`	año, trimestre, mes, semana, día, festivo
`dim_time` (dada)	`time_key`	—	`hour`, `minute`, `period_of_day`
`dim_customer` (SCD2)	`customer_key`	`customer_id`	`city`, `segment`, `income_band`, `valid_from`, `valid_to`, `is_current`
`dim_merchant`	`merchant_key`	`merchant_id`	`name`, `category`, `city`
`dim_payment_method`	`payment_method_key`	`method_name`	`method_type` (card, digital_wallet, bank_transfer)

No llegan al DW:

- `name` y `email` del cliente: identifican a la persona y no responden ninguna pregunta analítica (principio de minimización, Ley 1581 de 2012).
- `monthly_income_cop` exacto: es un dato financiero sensible. Se reemplaza por `income_band` (< 3 M, 3–6 M, 6–10 M, ≥ 10 M COP), que conserva su valor analítico.
- `method_id` (llave técnica) y el detalle de `fraud_events` (`event_id`, `pattern`, `detected_at`): solo pasa `risk_score` como medida. `status` se convierte en la medida `is_approved`.

Medidas de la tabla de hechos y si son aditivas, semiaditivas o no aditivas.

- `amount_cop`: **aditiva**. Se suma por cualquier dimensión.
- `is_fraud` e `is_approved`: banderas que, como 0/1, son **aditivas** (`SUM(is_fraud::INT)` da el número de fraudes). Las tasas que se derivan de ellas, como `fraud_rate_pct` en Q2, son **no aditivas**: se calculan como cociente de sumas, nunca sumando tasas.
- `fraud_risk_score`: **no aditiva**. Se agrega con AVG o MAX; sumar puntajes no tiene significado. Es `NULL` en las transacciones sin evento de fraude.
- **Semiaditivas: ninguna.** Un saldo no se suma en el tiempo; eso ocurre en hechos *snapshot*, no en uno transaccional.

Por qué surrogate keys en lugar de las llaves del OLTP.
(1) **SCD2:** un `customer_id` tiene varias versiones, así que no puede ser PK; cada versión necesita su propia llave. (2) **Independencia:** el DW no depende de que el OLTP conserve o reutilice sus llaves, y admite otras fuentes. (3) **Tamaño:** un `INT` de 4 bytes frente a un `VARCHAR(10)` achica hechos e índices y abarata los joins (182,5 M de filas al año).

Por qué SCD Tipo 2 para la dimensión cliente: ¿qué atributo puede cambiar y qué análisis se daña si se sobrescribe?
Pueden cambiar `segment` y `city`: el generador del curso simula clientes que *"se mudan de ciudad o cambian de segmento entre lotes"*. Si se sobrescribe (tipo 1), las transacciones antiguas quedan con el valor actual: un cliente con fraudes siendo `mass` que pasa a `premium` sube la tasa de `premium` y baja la de `mass`, y *"¿Qué segmento de clientes tiene mayor tasa de fraude?"* queda con una respuesta falsa. Con SCD2 cada cambio crea una versión y cada transacción apunta con `customer_key` a la vigente en su fecha.

Tipos de dato e índices (`star_schema.sql`). Surrogate keys `INT … IDENTITY`. El índice único parcial `ux_dim_customer_current … WHERE is_current` garantiza una sola versión vigente por cliente. Estos cuatro los fija el dato de origen:

Columna	Tipo	Por qué
`transaction_id` (hechos)	`VARCHAR(20)` PK	Dimensión degenerada: conserva el código del OLTP (`TXN0000000001`); un `IDENTITY` rompería la trazabilidad.
`fraud_risk_score` (hechos)	`DECIMAL(5,4)`	Mismo tipo que `fraud_events.risk_score`; con 2 decimales, 0,7884 quedaría en 0,79.
`income_band` (cliente)	`VARCHAR(20)`	Es una etiqueta de rango (`'3-6M'`), no un número; se calcula en la carga.
`ix_fact_*`	B-tree en las 5 FK	Para joins y filtros; su efecto se mide en la Parte D.

3. Comparación de rendimiento con planes de ejecución (Parte D)

Las consultas de `sql/olap_queries.sql` usan solo `dw.*` y dan los mismos resultados que las del OLTP en los dos volúmenes (`check_queries.py` 3/3; celda por celda y sin redondeo, ninguna diferencia). Tiempos: mediana de 5 `EXPLAIN ANALYZE`, caché caliente.

Ejecuta ambas versiones con `EXPLAIN ANALYZE` y compara: tiempo total, tipos de scan y joins, filas procesadas.

Consulta	Modelo	50K (ms)	500K (ms)	Scan de la tabla grande, 50K → 500K	Joins	Filas leídas 500K	OLAP/OLTP 50K → 500K
Q1	OLTP	8,96	50,67	Bitmap Heap (`idx_txn_created_at`) → Parallel Seq	1 Hash	500.025	**1,51× → 1,06×**
	OLAP	13,56	53,83	Seq → Parallel Seq	2 Hash	500.295	
Q2	OLTP	18,19	73,23	Seq → Parallel Seq	1 Hash	501.501	**0,99× → 1,01×**
	OLAP	17,98	74,18	Seq → Parallel Seq	1 Hash	501.501	
Q3	OLTP	5,82	31,17	Bitmap Heap (`idx_txn_created_at`)	—	21.905	**1,66× → 1,20×**
	OLAP	9,66	37,46	Seq → Parallel Seq	2 Hash	504.591	

En este montaje el modelo estrella **no es más rápido**: empata en Q2 y es más lento en Q1 y Q3. Hay tres razones:

1. **PostgreSQL guarda las filas completas (almacenamiento por filas).** La ventaja del esquema estrella aparece en motores columnares (Redshift, BigQuery, Snowflake), que leen solo las columnas que la consulta usa, o con la tabla de hechos particionada por fecha. Aquí `fact_transactions` y `transactions` se leen fila a fila con el mismo Seq Scan: las filas leídas son prácticamente las mismas en Q1 y Q2.
2. **El OLTP filtra la fecha con su propio índice; la estrella, a través de una dimensión.** `transactions.created_at` tiene índice, y en Q3 el OLTP lee solo las 21.905 filas de la semana. En la estrella el filtro está en `dim_date.full_date`: el optimizador lee las 500.000 filas de hechos y descarta en el Hash Join, porque no convierte ese filtro en un rango sobre `date_key`. Por eso Q3 es la que más pierde.
3. **El sobrecosto de la estrella es casi fijo y pesa menos a medida que crecen los datos.** Los joins extra son con dimensiones pequeñas (91 fechas, 1.440 minutos, 8 comercios): construir su tabla hash cuesta lo mismo con 50.000 que con 500.000 hechos. La tabla lo muestra: con 10 veces más datos la desventaja baja de 1,51× a 1,06× en Q1 y de 1,66× a 1,20× en Q3. En Q1 desaparece también la ventaja del índice del OLTP, que con 500.000 filas pasa a Parallel Seq Scan igual que la estrella.

Lo que la estrella sí resuelve es lo que el 3FN no puede: Q2 respeta la historia del cliente (SCD2) y Q3 no recalcula `EXTRACT()` en cada fila.

Experimenta con los índices de la tabla de hechos (TODO 5 de `star_schema.sql`): mide el plan con y sin cada índice. ¿Todos ayudan? ¿Por qué el optimizador a veces prefiere un *sequential scan*?
Se midieron 7 configuraciones en cada volumen: con los 5 índices, sin cada uno por separado y sin ninguno (`BEGIN; DROP INDEX …; ROLLBACK`). **Ninguno ayuda:** en las 7 el plan es el mismo `Seq Scan`, y ninguna de las tres consultas usa ningún índice. Q1 necesita el 30 % de la tabla y Q2 el 100 %, y los joins se resuelven con Hash Join, que lee la tabla una vez en vez de buscar fila por fila. Para ver por qué, se prohibió el sequential scan (`SET enable_seqscan = off`):

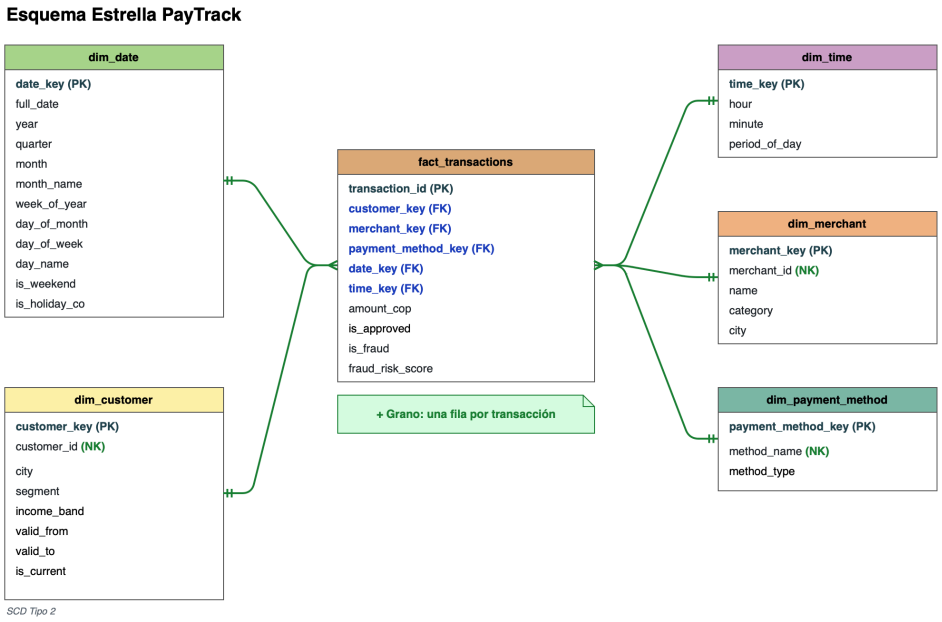
Consulta (fracción de la tabla)	Plan forzado	50.000: elegido → forzado	500.000: elegido → forzado
Q1 (30 %)	Bitmap Heap Scan `ix_fact_date`	15,3 → 15,3 ms	55,7 → **179,0 ms**
Q2 (100 %)	Index Scan `ix_fact_customer`	16,3 → **31,5 ms**	69,6 → **164,7 ms**
Q3 (4–5 %)	Bitmap Heap Scan `ix_fact_date`	8,6 → **3,6 ms**	36,3 → **25,1 ms**

El optimizador elige por costo estimado. Leer páginas contiguas cuesta `seq_page_cost = 1`, y un índice obliga a un acceso aleatorio por fila (`random_page_cost = 4`). Cuando se necesita una fracción grande de la tabla (Q1, Q2), el sequential scan es más barato, y forzar el índice multiplica el tiempo hasta por 3,2. En Q3, que es selectiva, el índice sí sería más rápido, pero el optimizador no lo ve. Hay dos causas: el filtro llega desde `dim_date` y no desde la tabla de hechos, y `random_page_cost = 4` supone un disco giratorio, no un SSD con los datos en caché.

Anexo (fuera de las 3 páginas): diagrama y carga

El enunciado pide el diagrama y los pantallazos como entregables propios; se incluyen aquí como referencia.

Diagrama del esquema estrella



Carga ELT del esquema estrella (Parte C, `load_star.py`)

Cada paso es un `INSERT … SELECT` de `oltp.*` a `dw.*`:

- **`dim_customer`:** una versión por cliente. `valid_from` es la fecha de alta, `valid_to` = `9999-12-31` (abierta), `is_current` = `TRUE`, e `income_band` sale de un `CASE` sobre `monthly_income_cop` (< 3 M, 3–6 M, 6–10 M, ≥ 10 M). No se cargan `name`, `email` ni el ingreso exacto.
- **`dim_payment_method`:** `method_type` es un atributo nuevo que no existe en el OLTP: `card`, `digital_wallet`, `bank_transfer`.
- **`fact_transactions`:** cada natural key se traduce a su surrogate key con un `JOIN`. El cliente se busca con el intervalo semiabierto `created_at::DATE >= valid_from AND < valid_to`, así que cada fecha cae en una sola versión. `date_key` (YYYYMMDD) y `time_key` (HHMM) salen de `created_at`, e `is_approved` = `status = 'approved'`. `risk_score` llega con `LEFT JOIN` a `fraud_events`: solo 200 de 50.000 transacciones tienen evento, y las demás quedan con `NULL` sin perderse.

Resultado en `main` (`817cc3d`): las 9 validaciones de `verify()` en verde, y `check_queries.py` 3/3.

```
PS C:\Users\santi\big-data-labs-2026-2> $env:PYTHONUTF8 =
  '1'; $env:DATABASE_URL = 'postgresql://postgres:postgres
@127.0.0.1:5432/paytrack'; git log --oneline -1; uv run p
ython labs/01-modelado/load_star.py
817cc3d (HEAD -> main, origin/main, origin/HEAD) Merge co
rrecciones-parte-b: Parte B y C corregidas, validaciones
en verde
✓ Esquema estrella creado
✓ dim_date: 91 filas
✓ dim_time: 1440 filas
✓ dim_customer: 500 filas
✓ dim_merchant: 8 filas
✓ dim_payment_method: 5 filas
✓ fact_transactions: 50000 filas

Validando la carga...
  ✓ dim_customer tiene un registro vigente por cliente
  ✓ dim_merchant tiene todos los comercios
  ✓ dim_payment_method tiene todos los métodos
  ✓ fact_transactions tiene una fila por transacción
  ✓ el monto total cuadra
  ✓ el número de fraudes cuadra
  ✓ el número de transacciones aprobadas cuadra
  ✓ toda transacción tiene una fecha y hora válidas en la
s dimensiones
  ✓ toda transacción tiene cliente, comercio y método en
las dimensiones

¡Esquema estrella cargado y validado!
PS C:\Users\santi\big-data-labs-2026-2>
```